

## 荷重センシングによる二度寝防止システムの開発

○班

提出日 2026 年 7 月 23 日

班番号 ○班

| 学籍番号 | 氏名 |
|------|----|
|------|----|

|          |      |
|----------|------|
| 20XXXXXX | 氏名 A |
|----------|------|

|          |      |
|----------|------|
| 20XXXXXX | 氏名 B |
|----------|------|

|          |      |
|----------|------|
| 20XXXXXX | 氏名 C |
|----------|------|

|          |      |
|----------|------|
| 20XXXXXX | 氏名 D |
|----------|------|

### 目次

| 番号 | 項目                 | 担当者  | ページ |
|----|--------------------|------|-----|
| 1  | 研究背景と目的            | 氏名 A | 2   |
| 2  | 提案システムの概要          | 氏名 B | 2   |
| 3  | ハードウェア・回路・ソフトウェア構成 | 氏名 B | 3   |
| 4  | 二度寝判定アルゴリズム        | 氏名 C | 6   |
| 5  | 実装内容               | 氏名 C | 7   |
| 6  | 模擬試験・実機確認と結果       | 氏名 D | 8   |
| 7  | 考察                 | 氏名 D | 11  |
| 8  | まとめと今後の課題          | 全員   | 11  |

本稿では、Raspberry Pi とロードセルを用いてベッド上の荷重変化を読み取り、起床後に再びベッドへ戻る行動を検出する二度寝防止システム「2dosumi」の開発について報告する。本研究では、二度寝を「離床後の再在床」という観測可能な状態遷移として定義し、荷重値のしきい値、監視時間、再在床の継続時間を組み合わせて判定する方式を採用した。模擬センサによる状態遷移の確認に加え、実機ではテスターによる電圧測定にもとづく不具合の切り分け、既知重量を用いたゼロ点・スケール校正、荷重変化の読み取りまでを確認した。

※提出前に、班番号・学籍番号・氏名・担当者名を実情報に置き換えること。

## 1 研究背景と目的

(担当：氏名 A)

朝の起床直後は、目覚ましを止めたあとに再び寝床へ戻る「二度寝」が起こりやすい。スマートフォンのアラームは音や振動によって覚醒を促すが、ユーザが一度アラームを止めてしまうと、その後に本当に起き続けているかを確認できない。そこで本研究では、寝床に戻ったという行動そのものを検出し、二度寝の直前または直後に再通知できるシステムを目標とした。

本研究の着眼点は、映像やマイクではなく荷重センサを用いることである。ベッドの脚やマットレス下にロードセルを設置すれば、利用者の在床・離床を、比較的プライバシーを保ったまま観測できる。画像認識に比べて得られる情報は少ないが、起床支援という目的に対しては「ベッド上にいるか」「離床後に戻ったか」という低次元の情報で十分である。既存の起床支援方法と本研究の位置づけを表 1 に整理する。

表 1 既存の起床支援方法と本研究の位置づけ

| 方式          | 長所         | 弱点           | 本研究との関係     |
|-------------|------------|--------------|-------------|
| スマートフォンアラーム | 導入が容易      | 停止後の行動を見ない   | 基本通知として併用可能 |
| カメラ監視       | 行動を細かく把握可能 | プライバシー負担が大きい | 本研究では採用しない  |
| ウェアラブル      | 身体状態を測定可能  | 装着忘れがある      | 非装着型を目指す    |
| 荷重センシング     | 在床変化を直接検出  | 設置・校正が必要     | 本研究の中心      |

研究目的は次の三点である。第一に、ロードセルから得られる荷重値をもとに在床状態を推定すること。第二に、離床後に一定時間以内で再び在床した場合を「二度寝リスク」として判定すること。第三に、判定結果を Web 画面や通知機構へつなげ、実用的な起床支援システムとして扱える形にすることである。

## 2 提案システムの概要

(担当：氏名 B)

提案システムは、ベッド下の荷重センサ、Raspberry Pi 上の判定プログラム、利用者が状態を確認する Web インタフェースおよび Android ネイティブアプリから構成される。センサは寝床の荷重を周期的に読み取り、プログラムはその値をしきい値と時間条件で解釈する。二度寝が疑われる状態になると、Android アプリがスマートフォン側でアラーム音と通知を出し、必要に応じて Webhook や Raspberry Pi 側ブザーも補助通知として利用できる。システムの全体構成を図 1 に示す。

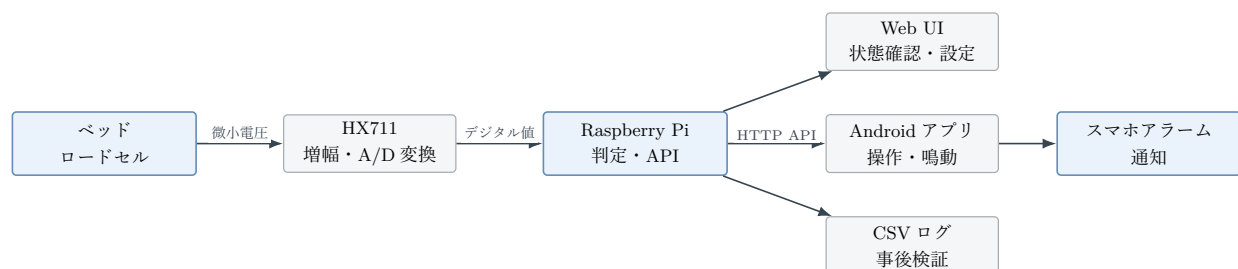


図 1 提案システムの全体構成

本システムが扱う状態は、主に「在床」「離床後監視」「再在床」「二度寝検出」であり、監視時間内に戻らなければ「起床成功」となる。重要なのは、単にベッドにいるかどうかではなく、起床予定時刻付近で一度離床

した後に戻るといった時間的な流れを扱う点である。このため、瞬間的なしきい値判定だけではなく、離床から再在床までの経過時間も判定に用いる。

Android ネイティブアプリは、Raspberry Pi 側の HTTP API に接続する日常利用の窓口として位置づけた。Web ブラウザでも状態確認は可能だが、利用者が自分から開く必要があるため、二度寝しかけている利用者へ情報を届ける手段としては受動的である。ネイティブアプリであれば Foreground Service と通知を用いて画面消灯中も状態を監視し、スマートフォン本体からアラーム音を鳴らせる。

起床直後には、枕元の端末から現在状態、重量値、次回アラーム、再確認状態を少ない操作で確認できる。さらに、音量、通知の有無、アラーム設定、二度寝確定までの秒数、再確認間隔、ゼロ点校正、既知重量によるスケール校正、センサ確認などを同じ画面から変更できる。このようにアプリは将来の拡張ではなく、通知を受け取り設定を調整するというシステムの利用体験そのものを担う要素である。

実装済みのプログラムでは、設定ファイルからセンサ種別、しきい値、通知方法、Web サーバ設定などを読み込める。これにより、実機ロードセルがない環境でも模擬センサを用いて判定ロジックを検証できる。また、設定値を変更することで、ベッドや利用者の体重差に応じた調整を行える設計とした。

### 3 ハードウェア・回路・ソフトウェア構成

(担当：氏名 B)

ハードウェア面では、Raspberry Pi、HX711、ロードセル、スマートフォンアプリによる通知を中心に構成する。ロードセルは微小なひずみを電気信号へ変換するセンサであり、HX711 はロードセル用の高分解能 A/D 変換器として広く使われている。Raspberry Pi は HX711 から得られる値を読み取り、Python プログラムで状態判定を行う。主要な構成要素を表 2 にまとめる。

表 2 主要構成要素

| 分類    | 要素                  | 役割                               |
|-------|---------------------|----------------------------------|
| センサ   | ロードセル               | ベッド上の荷重変化を測定する                   |
| 変換回路  | HX711               | ロードセル信号を Raspberry Pi で扱える値に変換する |
| 処理部   | Raspberry Pi        | 状態判定、通知、Web UI 提供を行う             |
| 通知部   | Android アプリ、通知、スマホ音 | 二度寝検出時に利用者へ知らせる                  |
| 表示部   | Web ブラウザ            | 現在状態、履歴、設定値を確認する                 |
| モバイル部 | Android ネイティブアプリ    | 状態確認、設定、校正、アラーム鳴動を行う             |

回路面で当初想定した標準構成は、4 個のロードセルを組み合わせたホイートストンブリッジである (図 2)。ロードセル内部のひずみゲージは、荷重を受けると抵抗値がわずかに変化する。しかし、その変化は非常に小さく、Raspberry Pi の GPIO へ直接入力しても読み取れない。そこで、4 個のロードセルをブリッジの 4 辺として構成し、左右の中間間に現れる差動電圧を HX711 で増幅・A/D 変換する。ブリッジ構成には、抵抗変化を差動電圧として取り出すことで感度を高められるうえ、温度変化のように 4 個の抵抗へ同じ向きに現れる変動を打ち消せるという利点がある。

実機検証では当初 4 個構成で組み、初期の読み取り不良は HX711 読み取り処理の実装ミスを修正することで解消し、4 個構成でもサンプル取得できることを確認した。その後、別日にロードセル 1 個のセンサ線を断線してしまったため、最終的な実機検証では故障したロードセルを外し、2 個の半橋ロードセルで 1 つのブリッジを構成する方式へ変更した。これは初期の読み取り不良とは別の事象であり、測定方式を単に簡略化したのではなく、実験を継続するための故障対応である。本研究の目的は体重を高精度に量ることではなく、在床、

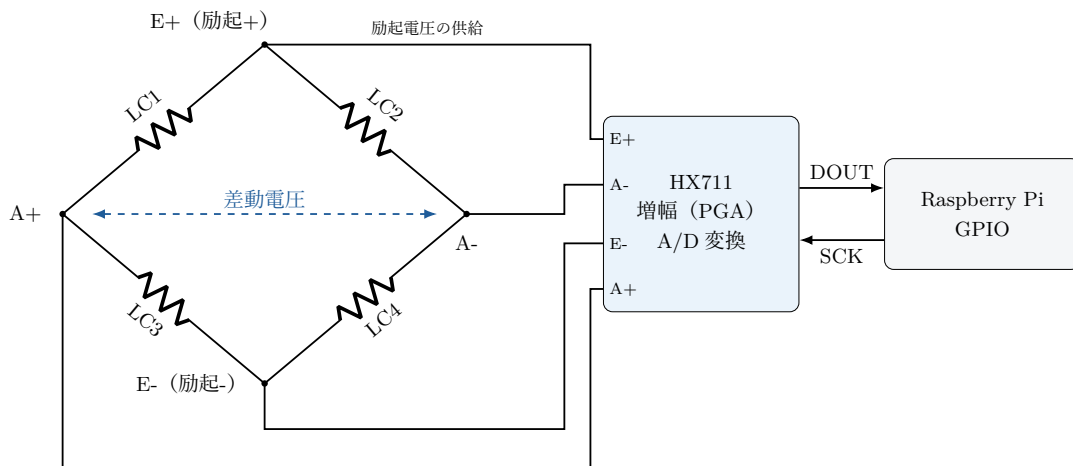


図2 4個のロードセル（LC1～LC4）で構成したホイットストンブリッジと HX711・Raspberry Pi の接続

離床、再在床の大きな荷重変化を判定することであるため、2 個構成でも判定ロジックの検証には十分であると判断した。実機検証で用いた 2 個ロードセル版の接続を図 3 と表 3 に示す。

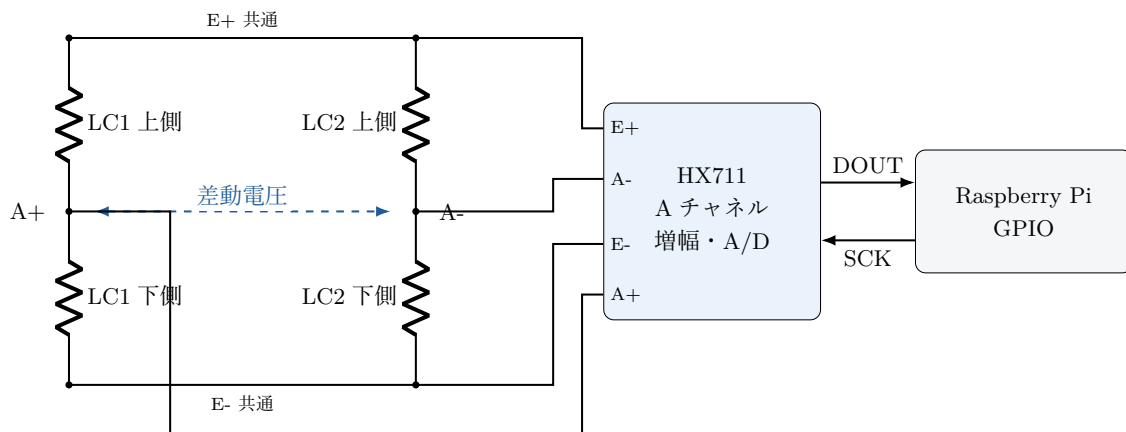


図3 実機検証で用いた 2 個の半橋ロードセルによるブリッジ構成

2 個のロードセルは向きや荷重のかかり方により、A+ と A- の符号が期待と逆になる場合がある。その場合でも故障とは限らず、既知重量を用いた校正でスケール係数の符号が決まり、ソフトウェア上では kg 相当値として扱える。本研究の実機でも、既知重量による校正後に荷重を kg 相当値として読み取れることを確認した（第 6 節）。

HX711 基板では、ロードセル側の端子は一般に E+、E-、A+、A- と表記される。E+ と E- は HX711 基板側からブリッジへ電圧を供給する励起端子であり、A+ と A- はブリッジの左右の中心から差動電圧を受け取る入力端子である。荷重がないときは左右の分圧がほぼ等しく、A+ と A- の電位差はほぼ 0 V になる。荷重がかかると一部のゲージが伸び、別のゲージが縮むため、片側の抵抗はわずかに増え、反対側はわずかに減る。励起電圧を  $V_E$ 、無荷重時の各辺の抵抗を  $R$ 、荷重による変化分を  $\Delta R$  とし、対角の 2 辺が  $+\Delta R$ 、残りの 2 辺が  $-\Delta R$  と逆向きに变化する理想的な場合を考えると、左右の分圧の差として現れる出力電圧は

$$V_{A+} - V_{A-} = \frac{\Delta R}{R} V_E$$

となり、抵抗の変化率に比例した電圧が得られる。実際の  $\Delta R/R$  はごく小さく、この差動電圧は数 mV 程度と微小であるため、HX711 は内部のプログラマブルゲインアンプ（チャンネル A では 128 倍）で増幅したうえで、24 ビットの A/D 変換器によりデジタル値へ変換し、DOUT・SCK の 2 線シリアルで Raspberry Pi へ送る（表 4）。

Raspberry Pi で使う場合は、GPIO が 3.3 V 入力である点に注意する。HX711 モジュールには 5 V 動作品も多く、DT 端子の High レベルが 5 V のままでは GPIO を傷めるおそれがあるため、本研究では HX711 を 3.3

表 3 2 個ロードセル構成での端子接続

| 端子        | 接続先                       |
|-----------|---------------------------|
| E+        | 2 個のロードセルの上側外端を共通接続       |
| E-        | 2 個のロードセルの下側外端を共通接続       |
| A+        | 1 個目ロードセルの midpoint       |
| A-        | 2 個目ロードセルの midpoint       |
| VCC       | Raspberry Pi の 3.3 V      |
| GND       | Raspberry Pi の GND        |
| DT / DOUT | Raspberry Pi の GPIO6 (D6) |
| SCK / CLK | Raspberry Pi の GPIO5 (D5) |

表 4 HX711 と Raspberry Pi の接続例

| 端子        | 接続先                        | 役割                       |
|-----------|----------------------------|--------------------------|
| E+ / E-   | ロードセルブリッジの電源端              | ブリッジ回路へ励起電圧を与える          |
| A+ / A-   | ブリッジの左右 midpoint           | 荷重で変化する差動電圧を入力する         |
| VCC / GND | Raspberry Pi の 3.3 V / GND | HX711 基板へ電源を供給する         |
| DT / DOUT | Raspberry Pi の GPIO6 (D6)  | 変換完了とデータ出力を受け取る          |
| SCK / CLK | Raspberry Pi の GPIO5 (D5)  | クロックを送り、データ読み出しとゲイン設定を行う |

V で駆動した。また、センサ配線は微小電圧を扱うため、長い配線や電源ノイズ、断線の影響を受けやすい。実機では、ロードセルの固定と荷重分散に注意し、ゼロ点補正と既知重量による校正で読み取り値を kg 相当へ変換する。

ソフトウェアは Raspberry Pi 側の Python プログラムと、スマートフォン側の Android ネイティブアプリに分けて構成した。Raspberry Pi 側では、センサ入力、判定器、通知、Web サーバを分離し、それぞれの責務を小さくした。これにより、実機センサから模擬センサへ切り替えても、判定器や表示側のコードを大きく変えずに試験できる。

Android アプリは Kotlin と Jetpack Compose により実装し、Retrofit/OkHttp を用いて、Raspberry Pi が公開する状態 API、設定 API、校正 API を呼び出す。センサ値を直接読まずに API 経由とすることで、ロードセルや HX711 に近い低レベル処理は Raspberry Pi に集約し、スマートフォン側は操作性と表示に集中できる。

### 3.1 筐体 (3D プリントケース) の設計

当初設計では、ロードセル 4 個、HX711、Raspberry Pi を一体化した体重計型の筐体を OpenSCAD で設計し、3D プリンタ (Bambu A1 mini、造形範囲 180 × 180 mm) で製作する構成を想定した。筐体はベッドの脚 1 本の下に敷き、その脚にかかる荷重の変化から在床・離床を読み取る。構成は、ロードセル・HX711・Raspberry Pi を収める下皿 (168 mm 角)、その上に浮かべる上蓋 (175.8 mm 角)、脚を受けるリング、および残り 3 本の脚の高さを揃えるスペーサー 3 個であり、全高は 34 mm である。設計の要点は荷重経路にある。脚の荷重は上蓋から裏面の 4 個のボスを介してロードセル中央の受圧ボタンのみに伝わり、下皿の支持台 (ペDESTAL) を経て床へ抜ける。上蓋は電子部品のどこにも接触せず (Raspberry Pi 最上部との隙間 2.6 mm)、外周のスカートは横ズレとほこりの侵入を防ぐだけで荷重を負担しない。これは市販の体重計と同じ構造であり、4 個構成では各ロードセルに荷重が分配される。設計した筐体の組立状態と断面を図 4 に示す。

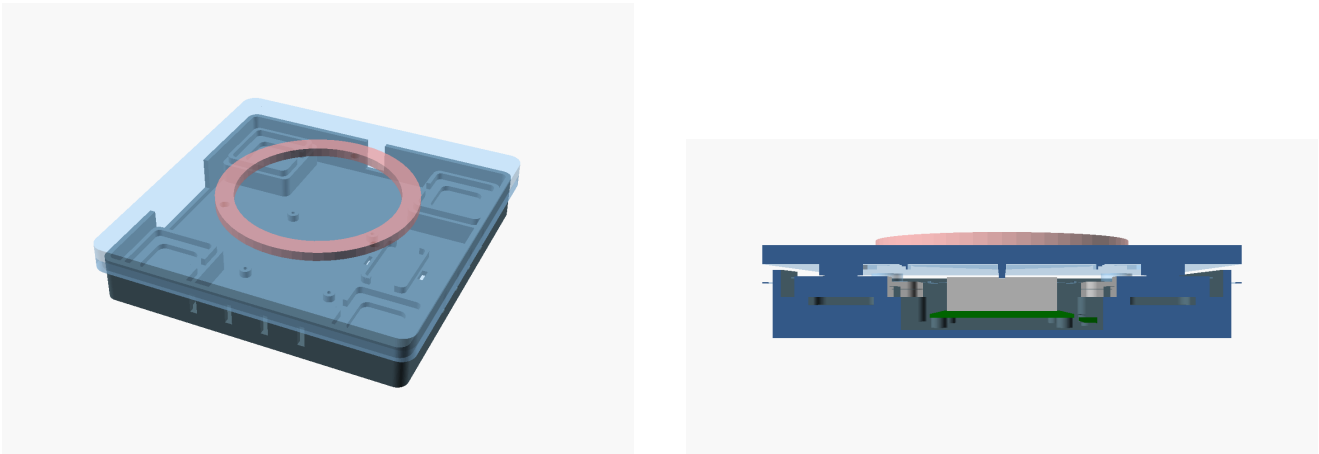


図 4 設計した筐体の組立ビュー（左）とロードセル中心線での断面（右）。断面では上蓋がロードセルの受圧ボタン 4 点のみに接触し、中央の Raspberry Pi（緑・灰）と隙間を保っていることが分かる

当初設計の耐久性は、荷重をほぼすべて「圧縮」で受ける構造とすることで確保した。3D プリント品は曲げや積層はく離に弱い一方、押し潰す方向には強いためである。ロードセルの定格は 4 個構成で  $50\text{ kg} \times 4 = 200\text{ kg}$  であり、就寝時にこの脚へかかる荷重（数十 kg）に対して十分な余裕がある。曲げを受ける唯一の部品は上蓋であり、板厚 7 mm に加えて裏面に深さ 5 mm のリブを配置し、Raspberry Pi の直上のみリブを省いてクリアランスを確保した。また、樹脂は常時荷重でわずかに変形が進む（クリープ）ため、長期間の使用ではゼロ点が徐々にずれ得るが、これは Web UI からのゼロ点校正で補正できる。製作時の主な印刷条件を表 5 に示す。曲げ剛性が必要な上蓋のみ壁数と充填密度を高め、圧縮しか受けない部品は標準設定として印刷時間を抑えた。

表 5 3D プリントの主な条件（材料は PETG、積層ピッチ 0.2 mm、サポート材なし）

| 部品         | 壁面層数 | 充填密度   | 補足                          |
|------------|------|--------|-----------------------------|
| 上蓋         | 4    | 25%    | トップ面・底面各 5 層。唯一曲げを受けるため強化する |
| 下皿         | 3    | 15%    | 床に全面接地するため曲げがかからない          |
| スペーサー（3 個） | 2    | 10～15% | 圧縮のみ。本体と同高にしてベッドを水平に保つ      |
| 脚受けリング     | 2    | 15%    | 脚の横ズレ防止用                    |

当初設計の組立では、ロードセルは下皿四隅のポケットにはめ込むだけで位置が決まり、上蓋のボスが上から押さえる。HX711 は中央のベイに収め、床の貫通スロットに通した結束バンドで固定する。Raspberry Pi は下皿のスタンドオフ 4 本（Pi 4 の取付穴ピッチ  $58 \times 49\text{ mm}$  に対応、下穴  $\phi 2.2\text{ mm}$ ）に M2.5×6 mm の小ネジをセルフタップでねじ込んで固定し、USB-C 電源ケーブルは側壁のスリットから外へ引き出す。設置面が沈むと支持台が傾き測定が不安定になるため、カーペット上への直置きは避け、フローリングか板の上に設置する。

4 二度寝判定アルゴリズム

（担当：氏名 C）

判定アルゴリズムは状態遷移として設計した。センサ値には揺らぎがあるため、単一のしきい値で在床・離床を切り替えると、境界付近で判定が細かく振動してしまう。そこで本システムでは、荷重値を設定した利用者体重に対する比率へ換算したうえで、離床の判定（体重比 0.3 未満）と再在床の判定（体重比 0.4 超）に異なるしきい値を用いるヒステリシスを持たせた。さらに、離床後の監視時間と、再在床の継続を確かめる確認時間を組み合わせることで、瞬間的な変動だけで二度寝と決めない設計とした。状態遷移の全体を図 5 に、判定に用いる主なパラメータを表 6 に示す。

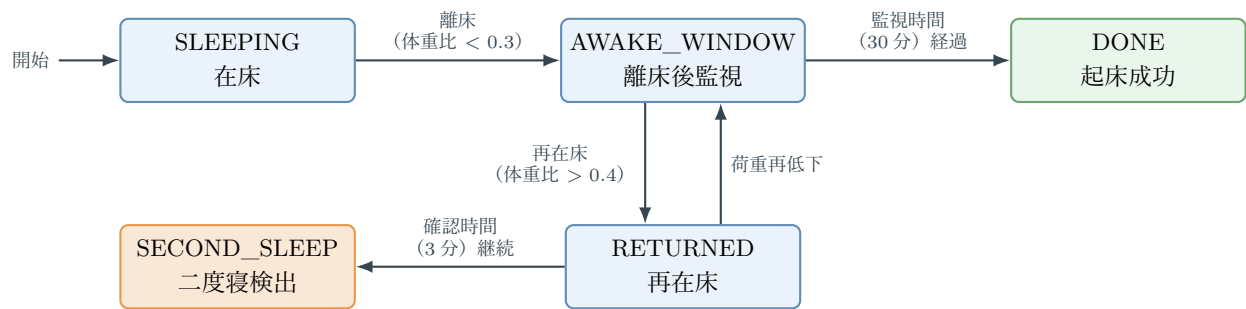


図5 二度寝判定の状態遷移（緑：正常終了、橙：二度寝検出。しきい値と時間は表6の既定値）

表6 判定に用いる主なパラメータ（既定値。設定ファイルで変更できる）

| パラメータ   | 既定値 | 意味                      |
|---------|-----|-------------------------|
| 離床判定比   | 0.3 | 体重比がこの値を下回ると離床とみなす      |
| 再在床判定比  | 0.4 | 離床後、体重比がこの値を超えると再在床とみなす |
| 監視時間    | 30分 | 離床後に再在床を見張る時間           |
| 確認時間    | 3分  | 再在床がこの時間続くと二度寝と確定する     |
| 中央値フィルタ | 9点  | 複数サンプルの中央値で1回分の読み値を決める  |
| 移動平均    | 5点  | 判定前に読み値の列を平滑化する         |

この方式の利点は、判定理由を説明しやすいことである。機械学習モデルを用いる場合、多様なデータを集めれば複雑な行動を分類できる一方、少数データでは過学習や説明困難性が問題になる。本研究段階では、まず比率のしきい値と時間条件による確実な基礎機能を作り、将来的にデータが蓄積した段階で学習型の判定へ拡張する方針とした。

処理の流れをまとめると、次のようになる。

1. センサ値を一定周期で取得し、中央値フィルタと移動平均で平滑化する。
2. 平滑化した値の体重比が離床判定比を下回ったら、離床時刻を記録して監視を始める。
3. 監視時間内に体重比が再在床判定比を超えたら、再在床状態へ移る。監視時間を過ぎれば、起床成功として監視を終える。
4. 再在床が確認時間続けば、二度寝検出として通知する。途中で荷重が再び下がった場合は監視状態へ戻る。

## 5 実装内容

（担当：氏名 C）

実装では、コマンドライン実行、Web サーバ、設定ファイル、ログ出力、Android アプリ、テストを用意した。コマンドラインからは通常実行と模擬実行を切り替えられる。Web サーバは現在の状態やイベント履歴を返す API を持ち、簡易画面から状態確認と設定変更ができる。ログは時刻、荷重値、状態、イベントを残すため、後から判定が妥当だったかを確認できる。Raspberry Pi 側のモジュール分割と依存関係を図6に、実装した主な機能を表7に示す。

模擬センサを用意した理由は、実機環境がなくても判定器の振る舞いを再現できるようにするためである。開発初期に実機だけへ依存すると、配線ミスやセンサ固定方法の問題と、ソフトウェアのバグが混ざってしまう。模擬入力で「期待する状態遷移が起こる」ことを先に確認しておくことで、実機試験時の切り分けが容易になる。

また、Web UI と Android アプリを持たせたことで、実験中にターミナルを見続けなくても現在状態を確認



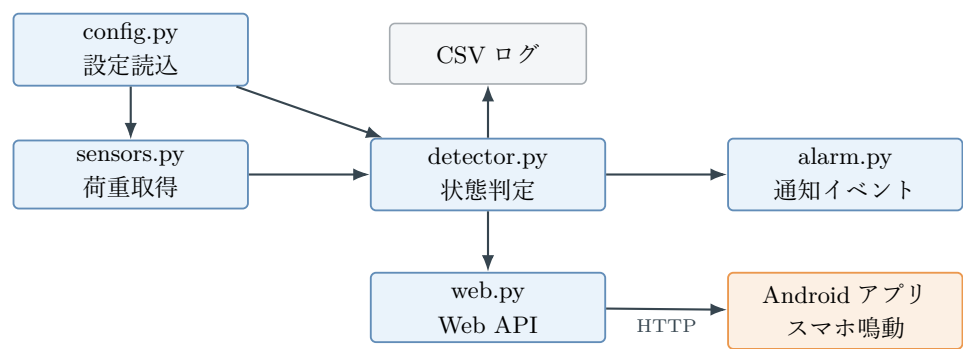


図 6 ソフトウェアのモジュール分割（青：Raspberry Pi 側の Python モジュール、橙：スマートフォン側、灰：データ）

表 7 実装した主な機能

| 機能          | 内容                                                |
|-------------|---------------------------------------------------|
| 設定読込        | JSON 設定から判定比、監視・確認時間、通知方法を指定する                    |
| センサ抽象化      | 実機 HX711 と模擬センサを切り替えられる                           |
| 平滑化         | 中央値フィルタと移動平均で読み値のノイズを抑える                          |
| 状態判定        | 離床、再在床、二度寝検出を状態遷移で処理する                            |
| 通知          | Android アプリのスマホ音、通知、Webhook、補助ブザーに対応する            |
| 定時アラーム      | 指定時刻に鳴動し、離床の継続を確認する起床ミッションを持つ                     |
| Web UI      | ブラウザから状態、ログ、設定を確認できる                              |
| Android アプリ | Kotlin/Jetpack Compose で状態確認、スマホアラーム、音量調整、校正操作を行う |
| 単体テスト       | 設定、センサ、判定、Web API の基本動作を確認する                      |

できる。Web UI は PC からの確認や発表デモに向き、Android アプリは起床直後に手元で操作する場面に向く。特に、スマートフォン側でアラーム一覧、センサ校正、音量、通知、二度寝確定秒数、再確認間隔を変更できるため、利用者が日常的に設定を変える運用を想定できる。アプリは Foreground Service で Raspberry Pi の状態 API を監視し、時刻アラーム、再アラーム、再入床、二度寝検出のイベントを受けてスマートフォン本体から音を鳴らす。これにより、Raspberry Pi にブザーを接続しなくても、枕元のスマートフォンを通知装置として利用できる。

6 模擬試験・実機確認と結果

（担当：氏名 D）

本節では、ソフトウェア上の判定ロジック、実機回路の電気的狀態、校正後の荷重読み取りの三段階で確認した結果を述べる。模擬試験では、実機の配線狀態に依存せず、状態遷移そのものが設計どおりに動くかを検証した。実機確認では、HX711 まわりの電源・励起・差動入力を測定し、読み取り不良の原因を配線とソフトウェアに分けて切り分けた。最後に、2 個ロードセル構成で校正を行い、在床・離床判定に必要な荷重差を読み取れるか確認した。結果の要約を表 8 に示す。

6.1 模擬センサによる状態遷移の確認

模擬試験では、体重 60 kg の利用者を想定し、在床狀態から一度離床し、その後ベッドへ戻る入力列を与えた。確認した実行では、18 サンプルの範囲で「在床」「離床後監視」「再在床」「二度寝検出」の順に狀態が遷移した。これにより、少なくとも設計した状態遷移がソフトウェア上で動作することを確認できた。与えた模擬



表 8 確認結果の要約

| 確認項目     | 方法                                  | 結果                                                  |
|----------|-------------------------------------|-----------------------------------------------------|
| 状態遷移     | 体重 60 kg 相当の模擬入力 18 サンプルを与える        | left_bed、returned、second_sleep_detected が順に 1 回ずつ発生 |
| 実機切り分け   | VCC、励起電圧、A+-A-、導通、DOUT の挙動を測定       | 電源・ブリッジ応答は確認でき、初期不具合は HX711 読み取り処理の実装ミスと判断          |
| 4 個構成の確認 | 読み取り処理修正後にサンプル取得を実施                 | 4 個ロードセル構成でも変換完了とサンプル取得を確認                          |
| 最終構成の校正  | 後日の断線対応として 2 個構成へ組み替え、ゼロ点・スケール校正を実施 | 既知荷重を 1%未満の誤差で読み取り、空床と在床相当の値が明確に分離                  |
| ソフトウェア試験 | 設定、センサ、判定、Web API の単体テストを実行         | 24 件のテストが通過                                         |

入力を図 7 に、確認したイベントを表 9 に示す。図中の 2 本の破線は、離床判定（体重比 0.3、荷重値 18 相当）と再在床判定（体重比 0.4、荷重値 24 相当）のしきい値であり、両者をずらしたヒステリシスによって境界付近の揺らぎで判定が振動しないようにしている。

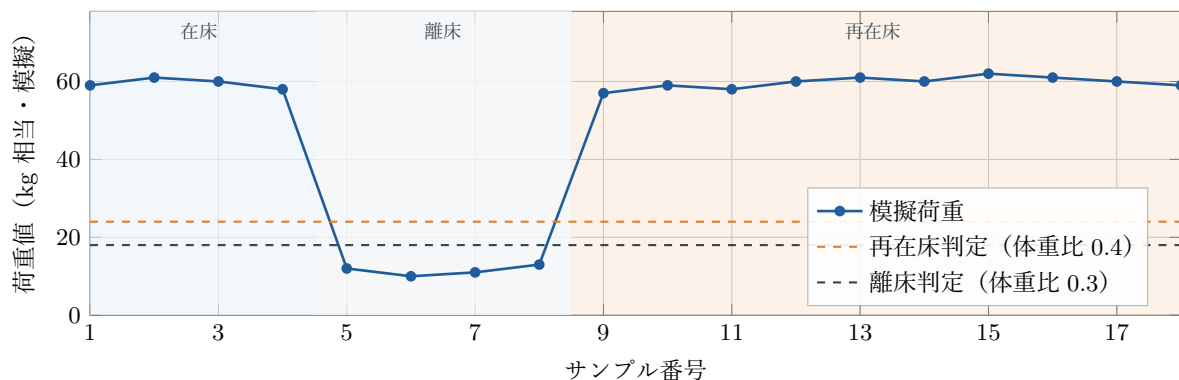


図 7 模擬入力における荷重変化と判定しきい値

表 9 模擬試験で確認したイベント

| 段階 | 状態           | 確認内容                 |
|----|--------------|----------------------|
| 初期 | SLEEPING     | 体重比が十分大きく在床と判定した     |
| 離床 | AWAKE_WINDOW | 荷重低下により離床後の監視へ移った    |
| 復帰 | RETURNED     | 監視時間内に荷重が戻ったことを検出した  |
| 検出 | SECOND_SLEEP | 再在床が継続し二度寝イベントを発生させた |

模擬試験の実行ログは CSV 形式で保存され、各行に時刻、センサ生値、kg 換算値、平滑化後の値、状態、イベントが記録される。ログ上でも left\_bed（離床）、returned（再在床）、second\_sleep\_detected（二度寝検出）の各イベントがこの順で 1 回ずつ出力されており、平滑化後の値がしきい値を横切ったサンプルで状態が切り替わることを行単位で追跡できた。この試験は繰り返し実行しても同じイベント列が得られ、判定器の動作は再現的であった。したがって、実機の測定誤差を含まない条件では、設計した状態遷移とイベント出力が意図どおり機能することを確認できた。

## 6.2 実機の電氣的測定と不具合の切り分け

実機は Raspberry Pi、HX711、ロードセルで構成し、まず当初設計どおりの 4 個構成で組み立てた。最初の動作確認では、HX711 の DOUT が変換完了 (Low) にならずタイムアウトする状態と、読み値が 24 ビット A/D の端点値 ( $\pm 8388607$ ) に張り付く状態が発生し、ソフトウェアからは有効な荷重値を得られなかった。端点値への張り付きは配線系の異常でも起こり得るため、当初は入力線の開放や断線を疑った。そこでテスターを用い、電源側から信号側へ向かって順に電圧と導通を実測して切り分けを行った。測定箇所と結果を表 10 に示す。

表 10 実機での電氣的測定とソフトウェア不具合の切り分け

| 測定箇所              | 期待される値       | 実測値           | 判断           |
|-------------------|--------------|---------------|--------------|
| HX711 の VCC-GND 間 | 3.3 V        | 3.3 V         | 電源供給は正常      |
| E+ - E- 間 (励起電圧)  | 電源電圧以下の一定値   | 約 2.68 V      | ブリッジへの励起は正常  |
| A+ - A- 間 (無荷重時)  | ほぼ 0 mV      | ほぼ 0 mV       | ブリッジの平衡は正常   |
| A+ - A- 間 (荷重印加時) | 数 mV 以下の微小変化 | わずかな変化を確認     | ブリッジ応答は存在    |
| センサ線の導通           | 全線で導通あり      | おおむね導通あり      | 初期不具合の主因ではない |
| HX711 読み取り処理      | 有効サンプルを取得    | 端点値・タイムアウトが発生 | 実装ミスを修正      |

この測定では、電源、励起、ブリッジの応答、センサ線の導通に大きな異常は確認できず、初期不具合の主因はセンサ線の断線ではなく、HX711 の読み取り処理におけるソフトウェア実装ミスであると判断した。該当箇所を修正した後は、4 個ロードセル構成でも DOUT が周期的に変換完了を示し、サンプル取得が成功した。

ただし、後日別の作業中にロードセル 1 個のセンサ線を断線してしまったため、最終的な検証では故障したロードセルを外し、2 個の半橋ロードセルによるブリッジ構成 (図 3) へ組み替えた。したがって、初期不具合の原因であったソフトウェア実装ミスと、2 個構成へ変更した理由である後日の断線は、別の事象である。

実測値の妥当性は、ロードセルの定格からも確認できる。使用したロードセルの定格出力を約 1 mV/V と仮定すると、実測した励起電圧 2.68 V では定格荷重時でも差動電圧は約 2.7 mV にとどまり、数十 kg の荷重では 1 mV 前後となる。これは「無荷重時はほぼ 0 mV、荷重印加でわずかに変化する」という実測結果と整合しており、HX711 が内蔵アンプで 128 倍に増幅してから A/D 変換する必要性を、実測からも裏付けられた。

## 6.3 校正と荷重読み取りの確認

組み替え後の 2 個構成で、ゼロ点校正とスケール校正を実施した。ゼロ点校正では、寝具を載せた空床状態で読み値を繰り返し取得し、その中央値をゼロ点とした。起動直後は値が安定しないため、ウォームアップとして最初の 5 サンプルを捨て、その後の読み値へ 9 点の中央値フィルタを適用している。スケール校正では、既知重量の荷重を載せた状態で生値を取得し、スケール係数を (荷重時の生値 - ゼロ点) ÷ 既知重量として算出した。

校正後に同じ既知荷重を載せ直して読み値を確認したところ、kg 相当値の誤差は約 0.8% (1%未満) であり、在床・離床判定には十分な精度で変換できることを確認した。また、在床相当の荷重を加えた状態と空床状態を繰り返して読み値を比較すると、両者は明確に分離しており、離床判定比 0.3・再在床判定比 0.4 のしきい値 (体重比換算) に対して十分な余裕があった。

この結果から、2 個ロードセル構成でも、在床と離床を区別するために必要な大きな荷重変化は取得できることを確認した。ただし、体重計としての絶対精度、寝返りや荷物による誤判定率、利用者ごとのしきい値最

適化は継続検証が必要である。単体テストとしては設定読込、センサ処理、Web API などの基本動作を確認し、現時点では 24 件のテストが通過している。そのため、本結果は「判定ロジック、ソフトウェア構成、実機読み取りの成立性」を示すものであり、最終的な実用性能を示すものではない。

## 7 考察

(担当：氏名 D)

本研究の有効性は、二度寝という行動を「起床後にベッドへ戻る」という観測可能な現象へ置き換えた点にある。眠気そのものを直接測ることは難しいが、ベッド荷重の変化なら安価なセンサで扱える。特に、カメラやマイクを使わないため、寝室で使う装置として心理的な抵抗を小さくできる可能性がある。

一方で、荷重だけに頼る方式には限界もある。たとえば、ベッドの上に荷物を置いた場合や、利用者がベッド端に腰掛けた場合、荷重の大きさだけでは在床と誤判定する可能性がある。また、ロードセルの設置位置によって測定値が変化するため、実機導入時には校正が不可欠である。今回の実機では、初期の読み取り不良はソフトウェア修正により解消し、4 個構成での動作を確認できた。一方で、後日にロードセル 1 個のセンサ線を断線したため、最終検証では 2 個構成へ変更した。これにより荷重分布の情報は少なくなったが、本研究で必要なのは体重の絶対値ではなく、空床状態と在床状態の差を安定して検出することである。2 個構成でも校正後に既知荷重を誤差 1%未満の kg 相当値として読み取れたため、在床・離床判定の基礎検証には有効であった。

この経験から、実機開発では回路図通りに配線するだけでなく、断線、接触不良、電源電圧、HX711 の応答、ソフトウェアの読み取り処理を順に切り分けることが重要だと分かった。特にロードセルは微小電圧を扱うため、線が細く、引っ張りや固定不足の影響を受けやすい。今後は、はんだ付け部の保護、ケース内でのケーブル固定、コネクタ化により、測定以前の故障要因を減らす必要がある。

ソフトウェア面では、状態遷移による設計が扱いやすかった。判定の流れが明確で、ログを見ればどの条件でイベントが起きたか説明できる。これは初期開発や授業発表に向いている。ただし、利用者の睡眠習慣が多様であることを考えると、将来的には曜日、起床予定時刻、過去の失敗履歴などを用いて、しきい値や監視時間を自動調整する仕組みが有効である。

## 8 まとめと今後の課題

(担当：全員)

本研究では、ロードセルによる荷重センシングを用いて、起床後に再びベッドへ戻る行動を検出する二度寝防止システムを開発した。Raspberry Pi 上で動作する Python プログラムとして、センサ入力、状態判定、ログ出力、通知イベント、Web UI を構成し、さらに Android ネイティブアプリから状態確認、設定変更、校正、スマートフォン側アラーム鳴動を行えるようにした。

得られた成果は三点に整理できる。第一に、模擬センサによる試験で、二度寝検出までの状態遷移とイベント出力を再現的に確認した。第二に、実機では電源電圧・励起電圧・差動電圧をテスターで実測し、初期の読み取り不良を配線不良ではなく HX711 読み取り処理の実装ミスとして切り分け、修正後に 4 個ロードセル構成でのサンプル取得を確認した。第三に、後日に発生したロードセル 1 個の断線へ対応するため最終検証では 2 個ロードセル構成へ変更し、既知重量によるゼロ点・スケール校正と荷重変化の読み取りを確認した。以上より、本研究の範囲では、判定ロジック、ソフトウェア構成、実機荷重読み取りの成立性を示すことができた。

今後の課題は、第一に実使用に近い条件での測定安定性評価である。ベッド構造、床面、センサ固定方法により値が変化するため、校正方法と設置方法を決める必要がある。第二に、誤検出を減らすことである。寝返り、荷物、短時間の着座などを区別するには、判定比・フィルタ・確認時間といったパラメータの調整が考えられる。第三に、ハードウェアの信頼性向上である。後日に発生したようなセンサ線切れを避けるため、断線しにくい固定、ケーブル保護、コネクタ化を進め、必要に応じて 4 個ロードセル構成へ復帰できるようにする。

第四に、通知方法の改善である。Android アプリのスマホアラームを中心に、通知権限、音量、再通知間隔、照明制御、家族への通知など、利用場面に応じた出力を選べると実用性が高まる。

導入ゼミナールでの自由研究としては、電気電子工学の要素であるセンサ計測、A/D 変換、組み込み処理、情報通信、ユーザインタフェースを一つの具体的な生活課題へ結びつけられた。今後、実機実験を重ねることで、より信頼性の高い起床支援システムへ発展させたい。

## 参考資料

1. Avia Semiconductor, “HX711 24-Bit Analog-to-Digital Converter (ADC) for Weigh Scales,” データシート.
2. Raspberry Pi Ltd, “Raspberry Pi Documentation” (2026 年 7 月参照) . <https://www.raspberrypi.com/documentation/>
3. 本プロジェクトのリポジトリ一式 (README、Raspberry Pi 側実装、Android アプリ実装、単体テスト) .